

Partie 09 — Fiabiliser et partager une analyse

Calculs fiables, environnement R et partage de code

Oskar Laverny

8 octobre 2026

Aix-Marseille Université

De mon ordinateur à celui d'un collègue



Vous avez déjà découvert Quarto : texte, code, résultats et rendu.

Aujourd'hui, nous allons vérifier que cette analyse :

- fonctionne depuis une session propre ;
- fonctionne dans un autre dossier ;
- décrit les packages utilisés ;
- produit des résultats vérifiés et peut être transmise.

Quatre questions différentes

Question	Moyen
Avec quelles données et quels choix ?	Données, code, paramètres, graine
Peut-on relancer le calcul ?	Session propre et chemins relatifs
Avec quelles versions des packages ?	<code>sessionInfo()</code> et <code>renv</code>
Le code fait-il ce que l'on attend ?	Contrôles et tests sur des cas connus

Quarto garde le texte, le code et les résultats ensemble.

Le résultat change : que vérifier ?

Une moyenne de TAS change entre deux analyses. Hypothèses :

- les données ont changé ;
- les valeurs manquantes sont traitées autrement ;
- un paramètre dépend de la console ;
- une version de package a changé.

Avant de conclure, retrouver les entrées, l'environnement et le code exact.

Fiabiliser le projet existant



Un projet dont on peut expliquer les fichiers

```
projet/  
├─ data/patients.csv  
├─ R/fonctions.R  
├─ rapport.qmd  
├─ README.md  
└─ resultats/
```

Données d'entrée et sources permettent de reconstruire les sorties. Conserver les données brutes ; effectuer les transformations dans le code.

Une dépendance cachée

```
# Dans la console seulement :  
seuil <- 140  
  
# Dans le rapport :  
mean(patients$tas >= seuil, na.rm = TRUE)
```

Un nouveau processus R ne connaît ni `patients` ni `seuil`. Importer les données et définir les paramètres dans le document.

Des chemins qui voyagent

```
patients <- readr::read_csv(  
  "data/patients.csv", show_col_types = FALSE  
)  
source("R/fonctions.R")
```

Pour cet atelier, garder `rapport.qmd` à la racine du projet et lancer le rendu depuis ce dossier. Un chemin relatif est interprété depuis un répertoire de travail : il faut aussi rendre cette convention explicite.

```
set.seed(2026)  
moyennes <- replicate(1000, mean(rnorm(30)))
```

Fixer la graine avant l'expérience permet de rejouer les tirages dans le même environnement. La graine ne remplace ni les données ni le code ; elle n'élimine pas les différences possibles entre environnements.

Retrouver l'environnement R



Installer et charger : deux opérations

```
install.packages("ggplot2") # installer dans une bibliothèque  
library(ggplot2)           # charger dans la session  
packageVersion("ggplot2") # connaître la version  
.libPaths()                # voir les bibliothèques utilisées
```

Installer aujourd'hui ne garantit pas d'obtenir la version utilisée il y a six mois. Les imports doivent être explicites dans l'analyse.

```
sessionInfo()
```

La sortie décrit notamment R, la plateforme et les packages chargés.

Elle aide au diagnostic, mais ne réinstalle rien et ne constitue pas à elle seule un inventaire de tous les packages nécessaires au projet.

Une bibliothèque globale peut être partagée par plusieurs analyses. Mettre à jour un package peut changer leur comportement.

`renv` associe au projet :

- une bibliothèque de packages ;
- un fichier `renv.lock` décrivant les versions ;
- des fichiers d'activation de cet environnement.

Les installations de packages d'autres projets restent indépendantes.

Initialiser un petit projet — exercice 4

Créer un dossier neuf `atelier_renv`, hors du dépôt du cours, et l'ouvrir comme projet.
Vérifier `getwd()` avant de lancer :

```
install.packages("renv") # une fois si nécessaire  
renv::init()  
renv::status()
```

`init()` prépare la bibliothèque et le verrouillage initial. La fiche fournit un script utilisant le petit package `digest` : l'installer dans ce projet, enregistrer le script, puis faire `snapshot()`.
Un accès Internet est nécessaire pour les téléchargements.

Après une modification des dépendances et un test du code :

```
renv::snapshot()
```

Sur un projet récupéré ailleurs, après activation :

```
renv::restore()
```

`snapshot()` met à jour le verrouillage ; `restore()` réinstalle les versions enregistrées. Ne pas remplacer le verrouillage reçu avant de le restaurer.

Qu'est-ce que le verrouillage décrit ?

Extrait illustratif d'un `renv.lock` :

```
{  
  "R": { "Version": "4.2.1" },  
  "Packages": {  
    "ggplot2": {  
      "Package": "ggplot2",  
      "Version": "3.4.0"  
    }  
  }  
}
```

Cet extrait n'est pas un verrouillage complet : `renv` enregistre aussi les sources et d'autres métadonnées. Le fichier est généré par l'outil.

À la fin de l'essai, pour désactiver `renv` dans ce projet :

```
renv::deactivate()
```

Cette commande retire l'activation automatique et redémarre la session. Si le redémarrage n'est pas automatique, redémarrer R puis vérifier `.libPaths()`. Par défaut, le verrouillage et la bibliothèque du projet sont conservés.

Pour réactiver plus tard : `renv::activate()`. Ne pas utiliser `clean = TRUE` pour ce simple essai : cela supprime le verrouillage et le dossier `renv/`.

Ce que `renv` ne restaure pas

`renv` gère les packages R et décrit la version de R utilisée.

Il ne réinstalle pas automatiquement :

- R lui-même ;
- Quarto ou les outils système ;
- les données absentes ;
- les choix effectués dans une ancienne console.

Documenter aussi ces prérequis dans le README.

Vérifier avant de partager



Un programme peut reproduire exactement le même résultat faux.

- **Reproductibilité** : retrouver les calculs et leur environnement.
- **Fiabilité** : vérifier que le code respecte les règles annoncées.
- **Validité** : vérifier que ces règles répondent à la question posée.

Une graine et un verrouillage ne corrigent pas une formule erronée.

Définir le contrat d'une fonction

Pour calculer une proportion au-dessus d'un seuil :

- entrée : un vecteur numérique et un seuil numérique unique ;
- les valeurs manquantes sont exclues ;
- sortie : un nombre entre 0 et 1 ;
- sans observation disponible : une erreur explicite.

Ce choix n'est pas universel : il doit être annoncé.

Refuser des arguments incohérents

```
proportion_seuil <- function(x, seuil = 140) {  
  stopifnot(is.numeric(x), is.numeric(seuil))  
  stopifnot(length(seuil) == 1L, is.finite(seuil))  
  observes <- x[!is.na(x)]  
  if (length(observes) == 0L) {  
    stop("Aucune observation disponible")  
  }  
  mean(observes >= seuil)  
}
```

Un arrêt explicite est préférable à un résultat inexploitable silencieux. Ici, les valeurs infinies de x restent autorisées par le contrat.

Un exemple dont on connaît la réponse

```
tas <- c(120, 145, NA, 160)
proportion_seuil(tas, 140) # 2/3

stopifnot(
  all.equal(proportion_seuil(tas, 140), 2 / 3),
  proportion_seuil(tas, 150) == 1 / 3
)
```

`stopifnot()` arrête l'exécution si une vérification échoue. `all.equal()` compare des nombres avec une tolérance.

Tester les limites

```
stopifnot(  
  proportion_seuil(c(100, 110), 140) == 0,  
  proportion_seuil(c(150, 160), 140) == 1,  
  proportion_seuil(140, 140) == 1  
)
```

À lancer séparément pour observer les erreurs attendues :

```
proportion_seuil(c(NA_real_, NA_real_))  
proportion_seuil(numeric(0))  
proportion_seuil(c("120", "160"))
```

Ne pas confondre manquant et négatif

```
tas <- c(120, 145, NA, 160)
mean(tas >= 140, na.rm = TRUE) # 2/3
sum(tas >= 140, na.rm = TRUE) / length(tas) # 2/4
```

Les deux codes répondent à des questions différentes. Le second conserve les valeurs manquantes dans le dénominateur.

Documenter la population utilisée, pas seulement la formule.

Des tests qui protègent les modifications

Après une réécriture, vérifier les résultats avant les performances :

```
x <- c(1, 2, 3, NA_real_)
methode_a <- mean(x, na.rm = TRUE)
methode_b <- sum(x, na.rm = TRUE) / sum(!is.na(x))
stopifnot(all.equal(methode_a, methode_b))
```

Un seul exemple ne prouve pas la correction générale. Ajouter des cas limites et des résultats calculés à la main.

Transmettre une analyse autonome



Le README est le mode d'emploi

Un collègue doit retrouver :

- la question, les données et leurs unités ;
- la règle pour les valeurs manquantes ;
- les versions de R et Quarto ;
- la restauration des packages avec `renv` ;
- la commande de rendu et les résultats attendus.

Une archive ZIP suffit pour transmettre le dossier complet, incluant ces informations.

Que mettre dans le dossier partagé ?

```
projet/  
├─ data/patients.csv  
├─ R/fonctions.R  
├─ tests/verifications.R  
├─ rapport.qmd  
├─ README.md  
├─ renv.lock  
├─ .Rprofile  
└─ renv/  
    ├─ activate.R  
    └─ settings.json
```

Conserver les fichiers cachés d'activation. Ne pas transmettre la bibliothèque installée ni les caches.

Restaurer n'est pas enregistrer

Le destinataire ouvre le projet et restaure les packages :

```
renv::restore()  
renv::status()
```

L'auteur teste une modification puis enregistre le nouvel environnement :

```
renv::snapshot()
```

Ne pas lancer `snapshot()` à réception : cela risque de remplacer les versions attendues par celles de la bibliothèque locale.

Avant d'envoyer du code:

1. Corriger chemins absolus et objets créés seulement dans la console.
2. Fixer une règle pour les valeurs manquantes.
3. Tester la fonction sur des cas connus.
4. Fixer la graine de la simulation.
5. Décrire l'environnement et écrire le README.
6. Transmettre le dossier à un voisin qui tente de reconstruire le PDF.

Partager ne signifie pas tout diffuser

- Vérifier que les données sont partageables.
- Ne pas inclure mots de passe, clés ni données personnelles.
- Éviter `.RData` et `.Rhistory` : pas un mode d'emploi du calcul.
- Fournir les sources ; un PDF seul ne permet pas de refaire l'analyse.

Si les données sont confidentielles : expliquer comment y accéder et fournir, si possible, un petit jeu fictif pour tester le code.

Vous pouvez simuler un partage de la manière suivante:

1. Copier le dossier transmis vers un nouvel emplacement.
2. Ouvrir une nouvelle session dans ce projet.
3. Vérifier R et Quarto, puis restaurer les packages avec `renv`.
4. Exécuter les tests et reconstruire le rapport.
5. Comparer les résultats annoncés.

`renv` fixe les dépendances R ; les tests contrôlent le comportement ; Quarto rassemble l'analyse et son explication.